

A SIMPLE, MINIMAL DSL FOR ASC

SMALL MODEL VS COMPILER VS CONSTRAINED DECODE

Date: 2026-08-22 (glossary and phrase unpacking added 2026-08-23)

Status: conversation capture / design instrument (not a spec, not an implementation plan)

Context: README § “ASC domain-specific language : *DSL* syntax” is the **only source of truth** for this syntax (the README is the only doc written entirely by the author). Revival v4 (Projet Complexe 2026 Revival (v4) - ASC, Projet Complexe and Projet Complexe ASC.md) for the agent / tool-boundary cut.

Origin: Cursor chat, 2026-08-22. Written out in full so the thread can be resumed from this file. Terms: **§8**. Unpacked phrases: **§2.2**, **§6**.

This note exists because the README DSL looks small enough to “just teach a model,” and that idea is easy to over-build — or to under-build, by skipping the one cheap piece that makes everything else safe.

The useful question is not only “is a small model possible?” It is:

Which job is neural (authoring DSL from natural language) and which job is symbolic (parsing, compiling, rejecting invalid or dangerous strings) — and which of those belongs in ASC core, which is a Fallback for tiny local models, and which should wait until the README’s own remaining TODOs are closed?

Terms that are easy to skip (PEG, GBNF, constrained decode, LoRA, gold pairs, ...) are defined in **§8**. The three sentences that were still opaque are unpacked in **§2.2** and **§6**, not only in the glossary.

0. WHAT THE README DSL ACTUALLY IS

The syntax in the README is a compact, **filename-safe** combinator language. It is meant to address argv the way `make` addresses entry points, and to appear in YAML `validate:` fields and in hook filenames such as:

```
*/$subject/transcribe-file(v-input_file_path).pre-index.hook.sh
```

The rewrite checklist still has **Stabilize DSL** unchecked. That means “write the parser / runtime that matches this README,” not “the README is one proposal among others.” The README is the language. A few lines inside it are still marked `TODO [wip]` (see §0.2); those are holes in the SoT, to be closed in the README.

“Parser” here is an ordinary program: it reads a DSL string and either rejects it or turns it into a tree (then into bash / argv). **PEG** and **recursive descent** are two ways to *write* that program (see §8.1). They are not a second language besides the README.

0.1 SURFACE

Entry points (as in `make`):

- `start = make start = asc/instance/start.sh`
- `service-rebuild = make service-rebuild = asc/extensions/compose/service/rebuild.sh`

Positional arguments:

- `@ = $@` (all arguments forwarded as-is)
- `p1 = $1, p2 = $2, ...`

Boolean options (shrink all `--` to `-` in prefixed syntax):

- `b-oneLine = --oneLine`
- `b-y = -y`
- `b-@ =` all boolean options forwarded (TODO [wip] YAGNI?)

Named options:

- `o-max=4 = --max=4 or --max 4 or -m 4` (TODO [wip] how to distinguish)
- `o-@ =` all named options forwarded (TODO [wip] YAGNI?)

Variables (any bash var in hook calling scope), `v-` prefix:

- `v-input_file_path = $input_file_path` in a `hook()` calling scope

Functions (whitelisted bash functions), `[]` enclosure:

- `[echo(a)] = echo "$@"`
- `[f_db_clear(fooBar)] = f_db_clear 'fooBar'`
- `[f_db_clear(v-DB_NAME)] = f_db_clear "$DB_NAME"`
- Nested / subshell `TODO [wip]: [f_db_clear(slug(p1))]` → `f_db_clear "$(slug 'fooBar')"` ?

Chaining:

- `[echo(v-baz)];echo(fooBar)` → `echo "$baz" ; echo 'fooBar'`
- `[echo(v-baz)];;echo(fooBar)` → `echo "$baz" && echo 'fooBar'`

Parallel:

- `[echo(v-baz)];-echo(fooBar)` → `echo "$baz" & echo 'fooBar' ; wait`

Piping:

- `[echo(v-baz)]+grep(fooBar)` → `echo "$baz" | grep 'fooBar'`

Conditional:

- `[echo(v-baz)]++[exit(1)] → echo "$baz" || exit 1`

Redirect:

- `[echo(v-baz)]--v-output_file_path → echo "$baz" > "$output_file_path"`
- `[echo(v-baz)]---v-output_file_path → echo "$baz" >> "$output_file_path"`

Loops:

- `[[echo(v-item)]v-foobar_arr] → for item in "${foobar_arr[@]}; do echo "$item"; done`
- `[[echo(v-item)]myfunc] → while read -r item; do echo "$item"; done < <(myfunc)`

“Normal” example:

```
transcribe-file(path/to/file.mp4)
```

→ `asc/extensions/transcription/transcribe/file.sh path/to/file.mp4`

YAML example (`foobar.entity.yml`, field `toto` must be slug or snake):

```
required:
  field:
    toto:
      validate: test-in(p1,slug(p1),snake(p1))
```

Any DSL starting with `test-*` is meant to compile to `[[ */test/*.sh ]] || exit 1.`

This is a **tiny combinator language**: entry points, a handful of prefixes (`p1` / `b-` / `o-` / `v-`), and a handful of operators (`;` `;;` `-;` `+` `++` `--` `---`, loops). That smallness is what makes both a parser and a small model *possible*. It is also what makes a neural interpreter the wrong default.

0.2 OPEN LINES STILL INSIDE THE README

Not a second syntax. These are the README’s own unfinished edges:

- `b-@` / `o-@` — YAGNI?
- `o-max-4` — `--max=4` VS `--max 4` VS `-m 4`
- nested calls — subshell or not (`[[f_db_clear(slug(p1))]]`)
- when YAML `validate:` runs (during entity validation?)

The parser should encode the README as written. Those TODOs stay explicit holes until they are decided **in the README**. There is no parser yet; “Stabilize DSL” is the work of matching this surface in code.

1. IS A SMALL MODEL POSSIBLE?

Yes. The language is tiny, regular, and combinatorial. A small model can emit it.

Revival v4 already named that path, and already limited it.

1.1 WHAT V4 ALREADY DECIDED

v4 §0.4 Refuse:

teaching the model a private DSL as its native function-call language

v4 §1.1:

DSL is a compact, filename-safe encoding of argv. It is for *addressing and validation*, not for chatting with a model.

v4 §1.3 table, “DSL instead of JSON/Python”:

No at the model boundary. Yes as ASC addressing. Compile YAML+DSL → JSON Schema / MCP tool list / TypeBox. Never the reverse as source of truth.

v4 §4.3, the precise split:

Teaching a 7B model to emit `o-max-4` as a function call is a **fine-tune / constrained-decode research project**, not a revival decision. Even then, the canonical description remains YAML.

Exception (small): for *tiny* local models with no native tool calling, `post_llm` can parse a **constrained** mini-format (even DSL-like) as a *Fallback Implementation* of “the model requested a tool.” Do not make that the protocol you show Claude.

v4 §5.3 mutually exclusive:

JSON Schema as *projection* vs DSL as the provider’s function-call language for frontier models — fights the ecosystem; split the Fallback for tiny local models only.

So: technically possible; classified as **research / Fallback**; refused as identity; YAML `able` stays source of truth for contracts; JSON Schema is the projection frontier models already know how to emit. The **DSL text** itself is specified only in the README.

1.2 TWO CONSTRAINTS BEFORE ANY TRAINING IS WORTH IT

1. **Close or encode the README's** `[wip]` **holes** (§0.2) before baking them into a model's weights. A LoRA adapter that guesses `o-max-4`'s meaning is inventing SoT (see §8.4).
2. **"Learn the syntax" is the wrong job for weights.** Syntax is a parser of the README (§8.1). The useful model job is **intent** → **valid DSL**, or **repair**, *under* that grammar.

If the goal is "a small model that speaks ASC," the cheap order is:

1. Write a parser (PEG or recursive descent, §8.1) that matches the README surface (and leaves §0.2 as explicit open productions until the README closes them).
2. Generate synthetic pairs from it (enumerate valid trees → captions — unpacked in §2.2).
3. **Constrained decode** (§8.2–8.3) so a 1–7B local model *cannot* emit invalid DSL.
4. LoRA only later, and only on that grammar, if evals show few-shot + constraints are not enough.

Fine-tune as v1 is what v4 / Yu & Yao refused: it is a *project*, not a toggle. Raschka's "train a local reasoner" door stays **late**, not day one.

1.3 FIRST FORK OFFERED IN THE CONVERSATION (A / B / C)

What "learn that syntax" could mean:

- **A.** A formal grammar + parser (no neural net) so ASC can compile DSL to bash.
- **B.** Constrained decode: a small local LLM may only emit valid DSL (v4 Fallback for models without tool calling).
- **C.** Fine-tune / LoRA a tiny model that authors DSL from natural language.

2. "THE DSL IS SO SIMPLE IT DOES NOT WARRANT A COMPILER" → THEREFORE C?

C is the right **authoring** idea. It is the wrong **replacement** for a compiler.

2.1 A COMPILER FOR THIS DSL IS NOT A COMPILER PROJECT

The "too simple for a compiler" read is half-right: nobody wants LLVM.

A compiler for this DSL is a short parser: tokenize prefixes, nested `()` / `[]`, then `;` `;;` `+` `++`. On the order of **100–300 lines**, whether written as recursive descent or generated from a PEG (§8.1). That is cheaper than gathering LoRA data (§8.4), and it is the only thing that can **reject** `rm -rf` dressed up as DSL. Revival v4 needs that gate: YAML `able` is canonical for contracts; the model never invents the tool surface; the model never sees `make hook`.

2.2 WHAT A TINY MODEL STILL CANNOT DO WITHOUT A PARSER

A tiny model that "knows the syntax" still emits **strings**. Without a parser you cannot:

- put DSL in `validate`: fields and trust the result
- generate or match hook filenames
- enforce an allowlist of entry points
- compile to bash deterministically
- produce training data that is *known* valid
- score a model (exact match after parse, not BLEU on the string — see below)

LoRA without a parser is training a model whose mistakes execute. LoRA that fills README `[wip]` holes in the weights is a second, unofficial SoT.

How the parser produces training data and scores the model

This is the sentence that was easy to misread:

The parser is also how you get training data (enumerate valid trees → English captions → LoRA pairs) and how you score the model (exact match after parse, not BLEU).

It is two factory jobs, both *after* you have a parser. Neither is "the model learns syntax by itself."

Job 1 — mint examples (the arrows). A grammar is a recipe for *trees*: `transcribe-file(path/to/file.mp4)` is a tree with root "call", child "entry point", child "path argument." Because the parser knows every legal shape, a small generator can walk those shapes and emit thousands of *valid* DSL strings (including pipes and loops if you allow them). For each string you write (or template) a caption in English / French / Portuguese: "transcribe this mp4." Each couple

natural language → that exact DSL string

is one **LoRA pair** (§8.4): the input you would show the model, and the output you want. "Gathering LoRA data" here mostly means *running that generator*, plus a small hand-written **gold** set (§8.5) you trust more than the synthetic bulk.

Job 2 — score without BLEU. After the model answers, do **not** ask "how many words overlap the reference?" That metric is **BLEU**: it was built for translation, and it rewards looking similar. Two problems for a DSL:

- `transcribe-file(a.mp4)` vs `transcribe-file(b.mp4)` look similar and score well, but they are different programs.
- `transcribe-file(a.mp4` (missing `)`) looks almost right and can still score decently, but it is illegal and must not run.

Exact match after parse means: run the parser on the model's string. If it fails, score = 0. If it succeeds, compare the **tree** (or the compiled bash) to the gold tree. Same meaning → pass, even if spacing differs. Wrong entry point or extra `+grep(...)` → fail. That is an eval you can automate; BLEU is not.

2.3 SO: DO C FOR NL → DSL, KEEP A TINY PARSER AS THE COMPILER

Skip LoRA at first. A 1–3B instruct model + constrained decode whose grammar is **the same parser grammar** already forbids invalid syntax (§8.2). LoRA only if that fails an eval set.

2.4 THREE SIZES

	What it is	Cost	When it wins
1. Parser only	DSL → bash; humans / YAML write DSL	An evening once the parser matches the README	Hook names, <code>validate:</code> , make. Needed anyway.
2. Parser + constrained decode (recommended path toward “small model”)	Existing tiny LLM may only emit valid DSL; parser executes	Grammar + a few dozen gold NL→DSL pairs (§8.5)	Local models with no JSON tool calling (v4 Fallback). No training run.
3. LoRA on top of 2	Specialist ~0.5–1B authors DSL from fr/en/pt	Dataset + evals + parser that matches the README	Only if 2’s error rate on combinators (pipes, loops) is actually bad.

Do not start at 3. Do not skip 1.
Size 2 is C’s *benefit* (natural language in, valid DSL out) without C’s *cost* (a training run, a specialist checkpoint, a second identity next to JSON tool calling). Size 3 is C done properly, and only after 1 and 2 exist.

3. WHY “SYNTAX LEARNING” AND “COMPILATION” ARE DIFFERENT JOBS

Job	Right tool	Failure mode if you use the other
DSL → bash / argv / hook path	Parser of the README (PEG or recursive descent; GBNF is the <i>same rules</i> in llama.cpp’s notation)	Model hallucinates operators; unsafe strings run
NL / intent → DSL	Small instruct model, optionally LoRA	Parser cannot guess what the human meant
Model → tool call (frontier)	JSON Schema projection of <code>able.yml</code>	Fighting every provider SDK; v4 refuse
Model → tool call (tiny local, no native tools)	Constrained mini-format, possibly DSL-like, parsed in <code>post_llm</code>	Making that the protocol you show Claude

A neural net that “learns the syntax” is trying to be a **parser with extra steps**. For a language this small, that is strictly worse than a 100–300 line parser: slower, non-deterministic, un-auditable, and useless for filename-safe hooks.
A neural net that **authors** DSL from natural language is a **translator**. That job is real. It still needs the parser on the output side, the way a programmer still needs `bash -n` or a compiler after Copilot.
Constrained decoding sits in the middle (§8.2): the weights propose the next token; the grammar forbids illegal tokens *while generating*; the parser still type-checks / allowlists / compiles. Same grammar in three roles (spec, decode mask, runtime compiler). That is the whole trick. You do not need three implementations of the language if the GBNF file is generated from the PEG (or is that grammar in another notation).

4. ALIGNMENT WITH REVIVAL V4 (DO NOT REOPEN CLOSED DOORS)

- Keep:
- README as the **only** specification of DSL text.
 - YAML `entity` / `able` as source of truth for **contracts**.
 - JSON Schema / MCP `tools/list` / `TextBox` as **projections** of those contracts.
 - DSL as ASC **addressing and validation** (argv, filenames, `validate:`).
 - `pre_llm` / `post_llm` as the harness wrap.
 - Tools as allowlisted entry points.
 - Tiny local models: constrained mini-format as **Fallback**, not as the Claude-facing protocol.
 - Routing / cascade among independently trained models (`2603.04445`): a 7B that emits JSON tool calls and a 0.5B that emits DSL would be two Technologies, not two sources of truth.
- Refuse:
- Teaching frontier models a private DSL as native function-call language.
 - Fine-tune as v1 / as identity.
 - Python/TS `@tool` registries that bypass YAML.
 - A LoRA (or any other doc) as a second specification of the syntax.
- Do not close:
- A later LoRA on the README grammar for a personal termbase / local authoring model (Yu & Yao: LoRA is a later door).
 - A later local reasoner (Raschka) — unrelated to *this* DSL, do not conflate.

The ASC-shaped gap in v4 §4.2 remains small: `llm` entry point + hooks; `able.yml` → JSON Schema; dispatcher from tool name → allowlisted entry point; traces. A specialist DSL-authoring model is **not** that gap. It is an optional Implementation of “the tiny local model requested a tool,” behind `post_llm`.

5. PRACTICAL ORDER OF WORK (WHEN RESUMED)

Not an implementation plan. A resume checklist.

1. Treat the README DSL section as the grammar. Close or explicitly keep the `[wip]` lines in the README (`b-e` / `o-e`, `o-max-4` form, nested subshell, when `validate` runs).
2. Write a parser (PEG or recursive descent, §8.1) + gold examples taken from that README section, including reject cases (`rm`, unquoted redirects, unknown entry points).
3. Compile DSL → bash / argv; use it for `validate` and for hook-stem checks. This is size 1. It is what “Stabilize DSL” on the rewrite checklist means.
4. Generate a GBNF (or Outlines) grammar from the same rules (§8.3). Few-shot a local 1–3B. Eval on held-out NL→DSL pairs. This is size 2.
5. Only if size 2 fails on combinators: synthetic dataset from the parser, then LoRA (size 3). Eval = AST exact match + allowlist violations = 0.

Until step 1’s `[wip]` holes are either closed in the README or marked as out-of-scope for the parser, a LoRA that fills them is inventing SoT.

6. OPEN QUESTION (RETURN HERE)

Two sentences that were doing too much work:

6.1 “MOST ENGLISH → ASC TRAFFIC IS ONE PIVOT + ARGS”

Most “English → ASC” traffic is one pivot + args (`transcribe-file` + a path). That is ordinary tool calling. JSON Schema already does it; the model does not need `+` `;;` `;-`.

Pivot here is the make-style name of *one* action (`transcribe-file`, `start`, `service-rebuild`). **Args** are its inputs (a path, a flag). What a human (or an agent) usually wants is:

```
Please transcribe path/to/file.mp4
```

That is already how ChatGPT / Claude / Ollama *tool calling* works: the model does not write bash. It emits a structured request, typically JSON:

```
{ "name": "transcribe-file", "arguments": { "path": "path/to/file.mp4" } }
```

JSON Schema is the contract for that JSON: “there is a tool named `transcribe-file`; it takes a string `path`.” Frontier models are trained to fill that form. ASC can *generate* that schema from YAML `able` (v4). The README DSL for the same request is just:

```
transcribe-file(path/to/file.mp4)
```

No pipe, no `;;` (and-then), no `;-` (parallel). Those operators are the **combinators** (`+` `;;` `;-` `++` loops). They are how you *compose* several entry points. If almost every request is a single pivot, teaching the model the combinators is teaching a language it will rarely use. JSON Schema already covers the common case.

6.2 “THE FULL COMBINATOR LANGUAGE IS A SMALL PROGRAMMING LANGUAGE”

The full combinator language (pipes, loops, filename hooks) is a small programming language. Teaching a 0.5B to write programs is real, and eval-heavy.

Combinators in the README are the glue operators, not the entry points themselves:

Operator	What it builds	Why this is “programming”
<code>+</code>	pipe	two processes, stdout → stdin
<code>;</code> / <code>;;</code>	sequence / and-then	order and short-circuit
<code>;-</code>	parallel	two processes + <code>wait</code>
<code>++</code>	or-else	failure path
<code>[[...]]</code>	loop	control flow
DSL in a <i>filename</i>	which hook wraps which call	composition in the filesystem

```
[transcribe-file(v-input_file_path)]+grep(hello)
```

is not a form with one field. It is a tiny program: run A, pipe into B. **Filename hooks** (`*/$subject/transcribe-file(v-input_file_path).pre-index.hook.sh`) are the same idea in a path: the *name* encodes wrap + args.
0.5B means a model with about 500 million parameters (TinyLlama-class): small enough to run locally, small enough that it will not “just know” how to write correct programs from a system prompt.

Eval-heavy means you cannot eyeball a few samples. You need many checks: does it parse? is the tree the intended program? did it invent `rm`? did it drop a pipe? A single-call JSON tool is one object to validate. A combinator string is a program; the test suite grows with every operator. That is why LoRA (if ever) belongs *after* a parser and a gold set, and only if you actually want models to *author* those programs rather than fill one tool form.

Which target should the tiny model author?

- **Single call:** natural language → one allowlisted entry point + args (JSON is enough; DSL is optional sugar).
- **Full combinators:** natural language → pipes, `;;`, loops, hook-stem DSL.

Until that is answered, default to size 1 (parser) for ASC core, and treat size 2 as the only model experiment worth a spike — and only as v4’s Fallback for tiny local models, not as the authoring UX for Projet Complexe.

7. POINTERS

- README § DSL (SoT): `/home/paul/Documents/asc/README.md`, from “ASC domain-specific language : DSL syntax”.
- Revival v4, especially §0.4 refuse list, §1.1–1.3, §4.3 “DSL instead of JSON”, §5.3 mutually exclusive row on JSON Schema vs DSL-at-the-provider.
- Related: Yu & Yao (language system, LoRA as later door); Raschka (train a reasoner: late door, not this DSL); Moslem & Kelleher 2603.04445 (routing/cascade, not a specialist syntax model as identity).

8. GLOSSARY (PLAIN LANGUAGE)

Nothing in this section is a second DSL spec. It only names the *tools* the rest of the note talks about. The language itself stays the README.

8.1 PEG VS RECURSIVE DESCENT — TWO WAYS TO WRITE THE SAME PARSER

A **parser** is a program that reads a string and either says “illegal” or builds a **tree** (often called an AST, abstract syntax tree). Example: `transcribe-file(a.mp4)` becomes something like `call(name=transcribe-file, arg=a.mp4)`. Compiling that tree to bash is a later, boring walk.

Recursive descent (also **recursive-descent parser**): you write that program *by hand*, one function per kind of thing.

```
parse_call()
→ read a name
→ if next char is '(', parse_args()
→ expect ')'
```

`parse_args` may call `parse_call` again when arguments nest (`slug(p1)`). That self-call is the “recursive.” “Descent” means you walk from the whole string down into smaller pieces. For the README DSL this is a short, readable bash or Python file.

PEG (Parsing Expression Grammar): the same idea written as *rules*, not as functions. A PEG is a text file that says, in a formal notation:

```
Call    <- Name '(' Args? ')'
Args    <- Expr '(' Expr)*
Expr    <- Call / Prefix / Literal
```

A PEG library turns those rules into a parser for you. PEG was designed so the rules map almost 1-to-1 onto recursive-descent functions (ordered choice: try this, else try that; no ambiguous “both are legal”).

“**Recursive descent or PEG**” in this note therefore means: *pick either handwritten functions or a grammar file* — both are ordinary parsers of the README. It does **not** mean two different languages. Use whichever is easier to keep in lockstep with the README. Size: still ~100–300 lines of real code either way.

A PEG is **not** a neural net. It never “guesses.” Same input → same tree or same error, every time.

8.2 CONSTRAINED DECODE

An LLM generates text **one token at a time** (a token is a chunk of text, often a word piece). At each step it assigns a probability to every token in its vocabulary, then samples.

Unconstrained: it can emit anything, including `transcribe-file(a.mp4` with no `)`, or `rm -rf /`, or English instead of DSL.

Constrained decode (constrained decoding / grammar-guided generation): at each step you **zero out** every token that would make the string so far illegal according to a grammar. The model still chooses *among the legal next tokens* (so it can still pick the wrong entry point if your grammar allows any name). It **cannot** produce a string that does not parse.

```
so far: transcribe-file(a.mp4
allowed next: )    or more path characters
forbidden:   +    ;    newline that closes the call ...
```

This is how size 2 works: the small model authors; the grammar is a muzzle; the parser still compiles and allowlists. No training run. You only need the grammar + a model that already speaks French/English/Portuguese.

Constraint is **syntax**, not **intent**. It will not stop `transcribe-file(/etc/passwd)` if a path is a legal argument. The allowlist / `able.yml` still has to reject that after parse.

8.3 GBNF, LLAMA.CPP, OUTLINES — THREE NAMES FOR “THE MUZZLE”

GBNF (GGML BNF, sometimes “GGML Backus–Naur Form”): the grammar file format used by **llama.cpp** (the common local-LLM runner behind many Ollama / CLI stacks). It looks like old-school BNF:

```
root ::= call
call ::= name "(" args? ")"
name ::= [a-z] [a-z0-9-]*
```

You pass that file to llama.cpp; generation is constrained decode (§8.2) against those rules. **llama.cpp GBNF** in this note just means “that stack”: local model + GBNF file. It is not a different theory from PEG. Ideally the GBNF is **generated from** the same PEG (or handwritten rules kept in sync), so there is one language, two notations: PEG for ASC’s compiler, GBNF for the local model’s muzzle.

Outlines (a Python library, often used with Hugging Face / vLLM): the same job — constrained decode — with JSON Schema or a grammar as the constraint, in Python rather than llama.cpp. The note says “**llama.cpp GBNF / Outlines**” as two *implementations* of size 2, not two designs. Pick one runtime: Ollama/llama.cpp → GBNF; a Python serving stack → Outlines (or equivalent).

JSON Schema constrained decode is how frontier tool calling is often *implemented* on the provider side: the model may only emit JSON that matches the tool schema. Size 2 is the same idea, with the README DSL as the schema instead of JSON.

8.4 LORA, LORA DATA, LORA PAIRS

A full **fine-tune** rewrites (potentially) all of a model’s weights. Expensive, easy to wreck the model’s general French/English, easy to overfit.

LoRA (Low-Rank Adaptation) is a *small add-on*: you freeze the base model and train a thin adapter (often a few megabytes). At run time: base model + adapter ≈ a specialist. Cheap to train on a laptop/GPU, easy to disable.

In this note LoRA always means: *teach a tiny local model to author README-DSL from natural language* (path C / size 3). It is not how ASC should *execute* DSL.

A **LoRA pair** (training example) is one couple:

```
input:  "Transcris le fichier data/media/talk.mp4"
output: transcribe-file(data/media/talk.mp4)
```

Gathering LoRA data means collecting many such pairs. Two sources:

- 1. **Synthetic (parser as factory, §2.2)**: enumerate legal trees → template a caption → you have a pair you *know* parses.
- 2. **Hand-written gold (§8.5)**: fewer, nicer, closer to how you actually speak.

Without (1) you will not have enough examples. Without (2) you only have stilted captions and you cannot trust the eval. The parser does not replace LoRA; it *feeds* it and *grades* it.

LoRA does **not** replace the parser. The adapter still emits a string; the parser still has to accept it. If you skip the parser, a fluent-looking `rm` wrapped in DSL-ish punctuation can execute.

8.5 GOLD NL→DSL PAIRS

NL = natural language (fr / en / pt). **DSL** = a README-legal string.

A **pair** is one NL utterance and the DSL you consider correct for it.

Gold (gold standard / ground truth) means a human treated this pair as *correct*, not merely “the generator emitted it.” Uses:

- **Few-shot**: paste 3–10 gold pairs into the prompt for size 2 (no training).
- **Eval**: hold some gold pairs out; after the model answers, parse and exact-match (§2.2 job 2).
- **LoRA**: mix gold + synthetic; never eval on the synthetic set you trained on.

“A few dozen gold NL→DSL pairs” in size 2 is that small hand-checked set — enough to prompt and score, not enough to train a specialist. Size 3 is when you add thousands of synthetic pairs *and still* score on gold (and on combinator cases, if you care about those).

Gold is **not** BLEU. BLEU would compare word overlap. Gold + parse compares trees.

8.6 ONE-LINE MAP BACK TO A / B / C

Term	Role in this note
Recursive descent / PEG	How you implement size 1 (the compiler)
GBNF / Outlines	How you muzzle a local LLM (size 2)
Constrained decode	The decoding algorithm that applies that muzzle
Gold NL→DSL pairs	The hand-checked examples for prompts and eval
LoRA / LoRA pairs / gathering data	Size 3: train a specialist, using parser-minted + gold pairs

Order stays: README → parser → (optional) constrained decode → (optional, later) LoRA.